1. Which keyword is used to declare and define a function in Python?

(A) function
(B) def
(C) func
(D) define

2. What character terminates the function header line in Python?

(A) {
(B) ;
(C) :
(D) ->

3. What value does a Python function return by default if it lacks an explicit return statement?

(A) 0
(B) False
(C) None
(D) Empty string

4. What is the difference between a parameter and an argument in Python?

(A) Parameters are defined in function headers; arguments are actual values passed during invocation
(B) Arguments are in headers; parameters are passed
(C) They are 100% synonymous with no distinction
(D) Parameters are always integers; arguments are strings

5. Which built-in module provides comprehensive mathematical functions like sqrt, sin, cos, and log?

(A) cmath
(B) math
(C) numeric
(D) calc

6. What does math.sqrt(25) return in Python?

(A) 5
(B) 5.0
(C) 25
(D) None

7. What does math.ceil(4.2) return?

(A) 4
(B) 5
(C) 4.0
(D) 5.0

8. What does math.floor(4.8) return?

(A) 4
(B) 5
(C) 4.0
(D) 5.0

9. Which keyword inside a function is used to declare that a variable refers to the global module scope?

(A) global
(B) extern
(C) public
(D) nonlocal

10. Which keyword introduced in Python 3 allows modifying a variable in an outer enclosing (non-global) scope?

(A) global
(B) outer
(C) nonlocal
(D) parent

11. What syntax is used to define variable-length positional arguments in a function definition?

(A) *args
(B) **kwargs
(C) &args
(D) ...args

12. What data structure does *args pack variable-length positional arguments into?

(A) list
(B) tuple
(C) dict
(D) set

13. What syntax is used to capture variable-length keyword arguments in a function definition?

(A) *args
(B) **kwargs
(C) &kwargs
(D) kw: dict

14. What data structure does **kwargs pack variable-length keyword arguments into?

(A) list
(B) tuple
(C) dict
(D) namedtuple

15. What is an anonymous, single-expression function called in Python?

(A) inline function
(B) lambda function
(C) macro
(D) static function

16. What is the correct syntax for a lambda function that squares a number x?

(A) lambda x -> x**2
(B) lambda x: x**2
(C) def lambda(x): return x**2
(D) lambda(x) { x**2 }

17. What is recursion in computer programming?

(A) A loop that never terminates
(B) A function that directly or indirectly calls itself
(C) Passing a function as an argument
(D) Compiling bytecode dynamically

18. What essential component must every recursive function have to prevent infinite execution?

(A) Global counter
(B) Base case (stopping condition)
(C) While loop
(D) Try-except block

19. What exception is raised when recursion exceeds Python's maximum recursion limit?

(A) StackOverflowError
(B) RecursionError (subclass of RuntimeError)
(C) MemoryError
(D) SystemError

20. What function in the sys module returns the current maximum recursion depth limit?

(A) sys.recursion_limit()
(B) sys.getrecursionlimit()
(C) sys.max_depth()
(D) sys.stack_depth()

21. What is the typical default recursion depth limit in standard CPython?

(A) 100
(B) 500
(C) 1000
(D) 65536

22. Which data structure does the runtime environment use to manage active function call frames?

(A) Queue
(B) Call Stack
(C) Heap
(D) Binary Search Tree

23. What does each stack frame (activation record) pushed onto the call stack store?

(A) Local variables, parameter values, and return address
(B) Global module variables
(C) Bytecode compiler tables
(D) Operating system drivers

24. What is a pure function?

(A) A function that uses only pure C extensions
(B) A function that produces no side effects and returns identical output for identical inputs
(C) A function without docstrings
(D) A lambda function

25. What is a higher-order function in Python?

(A) A function that runs with administrator privileges
(B) A function that accepts another function as an argument or returns a function
(C) A function containing nested classes
(D) A function with > 100 lines of code

26. Which built-in higher-order function applies a function to all elements of an iterable?

(A) filter()
(B) map()
(C) reduce()
(D) apply()

27. Which built-in higher-order function extracts elements of an iterable for which a predicate returns True?

(A) map()
(B) filter()
(C) extract()
(D) select()

28. From which module in the standard library must reduce() be imported in Python 3?

(A) itertools
(B) functools
(C) operator
(D) math

29. What rule defines the order of variable scope lookup in Python?

(A) LIFO Rule
(B) LEGB Rule (Local, Enclosing, Global, Built-in)
(C) FIFO Rule
(D) Static Inheritance Rule

30. What is the output of def f(): pass; print(f())?

(A) 0
(B) None
(C) pass
(D) Error

31. What is the mathematical base case for the factorial function n!?

(A) 0! = 1
(B) 0! = 0
(C) 1! = 2
(D) Factorial has no base case

32. What is the recurrence relation for the Fibonacci sequence: F(n) for n >= 2?

(A) F(n) = F(n-1) * F(n-2)
(B) F(n) = F(n-1) + F(n-2)
(C) F(n) = 2*F(n-1)
(D) F(n) = F(n-1) + n

33. What are the base cases for the standard Fibonacci sequence F(0) and F(1)?

(A) F(0) = 0, F(1) = 1
(B) F(0) = 1, F(1) = 1
(C) F(0) = 0, F(1) = 0
(D) F(0) = 1, F(1) = 2

34. What is the minimum number of moves required to solve the Tower of Hanoi puzzle with n disks?

(A) 2*n
(B) 2^n - 1
(C) n^2
(D) n!

35. What does math.pow(2, 3) return in Python?

(A) 8
(B) 8.0
(C) 6
(D) 9.0

36. What does math.factorial(5) return?

(A) 120
(B) 24
(C) 720
(D) 120.0

37. What does math.gcd(48, 18) return in Python?

(A) 6
(B) 3
(C) 2
(D) 9

38. What does math.radians(180) return?

(A) 3.141592653589793 (pi)
(B) 180.0
(C) 1.0
(D) 0.0

39. What does math.degrees(math.pi) return?

(A) 180.0
(B) 360.0
(C) 90.0
(D) 0.0

40. What happens if a function tries to read a global variable without declaring global x?

(A) SyntaxError
(B) It reads the global variable value successfully
(C) Raises UnboundLocalError
(D) Variables cannot be read

41. What is the classic pitfall of: def append_to(element, target_list=[]): target_list.append(element)?

(A) SyntaxError
(B) The default list is instantiated ONCE at function definition time and shared across calls!
(C) A new list is created on every call
(D) target_list is immutable

42. How should default mutable arguments (like lists or dicts) be idiomatically defined?

(A) def f(lst=[]):
(B) def f(lst=None): if lst is None: lst = []
(C) def f(lst=list()):
(D) def f(lst=copy([])):

43. What occurs when executing:\nx = 10\ndef foo():\n print(x)\n x = 20\nfoo()

(A) Prints 10 then sets x = 20
(B) UnboundLocalError: cannot access local variable 'x' where it is not associated with a value
(C) Prints 20
(D) NameError: global x not found

44. Why did the code in Q43 raise UnboundLocalError?

(A) Because assignment 'x = 20' causes the compiler to flag 'x' as a local variable throughout the entire scope!
(B) Because x is immutable
(C) Because print() cannot display integers
(D) Because functions cannot access globals

45. What is the output of:\ndef calc(a, b=10, c=20):\n return a + b + c\nprint(calc(5, c=30))

(A) 45
(B) 65
(C) 55
(D) TypeError

46. What is the output of:\ndef show(*args):\n return sum(args)\nprint(show(1, 2, 3, 4))

(A) 10
(B) (1, 2, 3, 4)
(C) 4
(D) TypeError

47. What is the output of:\ndef greet(**kwargs):\n return kwargs.get('name', 'Guest')\nprint(greet(role='admin', name='Aarav'))

(A) Guest
(B) Aarav
(C) {'name': 'Aarav'}
(D) TypeError

48. What does the special forward slash (/) in a function signature def f(a, b, /, c): enforce?

(A) a and b MUST be passed as positional-only arguments
(B) c must be positional-only
(C) Division is performed on arguments
(D) Arguments must be floats

49. What does the bare asterisk (*) in def f(a, *, b, c): enforce in Python 3?

(A) b and c MUST be passed as keyword arguments
(B) a must be a keyword argument
(C) Arbitrary arguments are accepted
(D) Multiplication is performed

50. What is the output of:\nsq = lambda x: x ** 2\nprint(list(map(sq, [1, 2, 3, 4])))

(A) [1, 4, 9, 16]
(B) [2, 4, 6, 8]
(C) 16
(D) TypeError

51. What is the output of:\nprint(list(filter(lambda x: x % 2 == 0, [1, 2, 3, 4, 5, 6])))

(A) [1, 3, 5]
(B) [2, 4, 6]
(C) [False, True, False, True, False, True]
(D) [2, 4]

52. What is the output of:\nfrom functools import reduce\nprint(reduce(lambda a, b: a * b, [1, 2, 3, 4]))

(A) 10
(B) 24
(C) [1, 2, 6, 24]
(D) 4

53. What is the time complexity of the naive unmemoized recursive Fibonacci function: fib(n) = fib(n-1) + fib(n-2)?

(A) O(N)
(B) O(2^N) or O(1.618^N) exponential time
(C) O(N^2)
(D) O(log N)

54. What is the auxiliary space complexity of naive recursive Fibonacci fib(n) on the call stack?

(A) O(1)
(B) O(N) stack frames depth
(C) O(2^N)
(D) O(log N)

55. What decorator in functools automatically memoizes function return values to eliminate redundant recursive calculations?

(A) @functools.lru_cache(maxsize=None)
(B) @functools.cache
(C) @functools.memoize
(D) Both A and B

56. What is the output of:\ndef outer():\n x = 10\n def inner():\n nonlocal x\n x += 5\n inner()\n return x\nprint(outer())

(A) 10
(B) 15
(C) 5
(D) UnboundLocalError

57. What is a closure in Python?

(A) A function that has terminated and freed its memory
(B) An inner function that retains access to variables in its enclosing lexical scope even after the outer function has returned
(C) A private class method
(D) A lambda function inside a loop

58. What is the output of:\ndef make_multiplier(n):\n return lambda x: x * n\ntriple = make_multiplier(3)\nprint(triple(10))

(A) 30
(B) 13
(C) 3
(D) TypeError

59. What is the output of:\ndef f(n):\n if n <= 1: return 1\n return n * f(n - 1)\nprint(f(4))

(A) 24
(B) 12
(C) 10
(D) 4

60. What is the output of:\ndef count_down(n):\n if n == 0: return\n print(n, end=' ')\n count_down(n - 1)\ncount_down(3)

(A) 3 2 1
(B) 1 2 3
(C) 3 2 1 0
(D) Infinite loop

61. What is the output of:\ndef print_up(n):\n if n == 0: return\n print_up(n - 1)\n print(n, end=' ')\nprint_up(3)

(A) 3 2 1
(B) 1 2 3
(C) 0 1 2 3
(D) 3

62. What is the output of:\ndef sum_digits(n):\n if n == 0: return 0\n return (n % 10) + sum_digits(n // 10)\nprint(sum_digits(1234))

(A) 10
(B) 24
(C) 4321
(D) 4

63. What does the following recursive function compute:\ndef mystery(a, b):\n if b == 0: return 0\n return a + mystery(a, b - 1)

(A) a + b
(B) a * b (Multiplication via repeated recursive addition)
(C) a ^ b
(D) a / b

64. What does the following recursive function compute:\ndef p(x, n):\n if n == 0: return 1\n return x * p(x, n - 1)

(A) x * n
(B) x ** n (Exponentiation via repeated multiplication)
(C) x + n
(D) n!

65. What is the output of:\ndef rev(s):\n if len(s) <= 1: return s\n return rev(s[1:]) + s[0]\nprint(rev('python'))

(A) 'python'
(B) 'nohtyp'
(C) 'p'
(D) 'y'

66. What is the output of:\nfuncs = [lambda x: x + i for i in range(3)]\nprint([f(10) for f in funcs])

(A) [10, 11, 12]
(B) [12, 12, 12] (Late binding closure gotcha: i evaluates to 2 at call time!)
(C) [10, 10, 10]
(D) [11, 12, 13]

67. How is the late binding closure trap in Q66 resolved so each function captures the current loop value?

(A) Use default argument: lambda x, i=i: x + i
(B) Use global i
(C) Use nonlocal i
(D) Use while loop

68. What does the inspect module in Python provide for functions?

(A) Memory profiling only
(B) Introspection of function signatures, parameters, source code, and frame stacks
(C) JIT compilation
(D) Disassembly to assembly language

69. What attribute holds the docstring of a user-defined function my_func?

(A) my_func.__doc__
(B) my_func.__help__
(C) my_func.doc
(D) my_func.__info__

70. What attribute holds the string name of a function object?

(A) my_func.__name__
(B) my_func.__id__
(C) my_func.name
(D) my_func.__str__

71. What attribute holds the dictionary of annotations (type hints) for a function?

(A) my_func.__annotations__
(B) my_func.__types__
(C) my_func.__hints__
(D) my_func.__signature__

72. Does Python enforce type annotations (e.g. def f(x: int) -> str) at runtime by default?

(A) Yes, it raises a TypeError if an incorrect type is passed
(B) No; Python type annotations are informational metadata and not enforced at runtime
(C) Yes, in Python 3.10+
(D) Only for floats

73. Which external static type checker tool is widely used in Python to verify type hints?

(A) mypy
(B) pylint
(C) pytest
(D) black

74. What is the output of:\ndef f(a, b=2, *args, **kwargs):\n return len(args) + len(kwargs)\nprint(f(1, 2, 3, 4, x=10, y=20))

(A) 2
(B) 4 (args has (3, 4) length 2; kwargs has {'x': 10, 'y': 20} length 2)
(C) 6
(D) 0

75. What is the output of:\ndef gcd(a, b):\n return a if b == 0 else gcd(b, a % b)\nprint(gcd(48, 18))

(A) 6
(B) 18
(C) 48
(D) 3

76. What is Tail Recursion?

(A) A recursive call where the function performs additional calculations after the return
(B) A recursive call that is the absolute final operation performed before returning
(C) A function that calls two subproblems
(D) Recursion with a while loop

77. Does standard CPython perform Tail Call Optimization (TCO) or Tail Call Elimination?

(A) Yes, Tail Call Optimization is automatically enabled at -O2
(B) No; Guido van Rossum deliberately rejected TCO to preserve full stack traces for debugging
(C) Yes, in Python 3.11+
(D) Yes, for integers only

78. Why did Guido van Rossum reject Tail Call Elimination in Python?

(A) CPython cannot handle jumps
(B) TCO destroys stack frame information, impairing stack traces during exception debugging
(C) TCO violates PEP 8 formatting
(D) TCO causes memory leaks in C

79. How can a tail-recursive algorithm be prevented from exceeding the recursion limit in Python?

(A) Convert it to an iterative while loop or use a trampoline pattern
(B) Increase sys.setrecursionlimit to 10 billion
(C) Use a lambda function
(D) Compile with PyPy only

80. What is the danger of setting sys.setrecursionlimit(1_000_000) on Windows/Linux?

(A) Python bytecode slows down
(B) Exceeding the physical OS C-stack memory limit causes an uncatchable segmentation fault / hard crash!
(C) Garbage collector is disabled
(D) Global variables are cleared

81. What is the time complexity of computing power x^n using Binary Exponentiation (Exponentiation by Squaring)?

(A) O(N) linear time
(B) O(log2 N) logarithmic time
(C) O(N^2)
(D) O(1)

82. What is the recurrence relation for Binary Exponentiation when n is even?

(A) P(x, n) = P(x, n/2)^2
(B) P(x, n) = x * P(x, n - 1)
(C) P(x, n) = 2 * P(x, n/2)
(D) P(x, n) = P(x^2, n)

83. What is Memoization?

(A) Formatting docstrings into HTML
(B) An optimization technique that caches the results of expensive function calls based on input arguments
(C) Translating Python into C++
(D) Deleting unused variables from stack

84. With memoization, what does the time complexity of recursive Fibonacci fib(n) reduce to?

(A) O(2^N) exponential
(B) O(N) linear time
(C) O(N^2) quadratic
(D) O(N!) factorial

85. What is the time complexity of the Tower of Hanoi algorithm with N disks?

(A) O(N)
(B) O(2^N) exponential time
(C) O(N^2)
(D) O(N!)

86. In the Tower of Hanoi, how many total moves are required to transfer 5 disks?

(A) 10
(B) 31 (2^5 - 1 = 31)
(C) 25
(D) 32

87. What is the time complexity of Binary Search implemented recursively on a sorted array of size N?

(A) O(1)
(B) O(log2 N)
(C) O(N)
(D) O(N log N)

88. What is the space complexity of recursive Binary Search due to call stack frames?

(A) O(1)
(B) O(log2 N) auxiliary stack frames
(C) O(N)
(D) O(2^N)

89. Can Python functions have arbitrary attributes assigned to them dynamically: my_func.custom_cache = {}?

(A) No, functions are immutable code objects
(B) Yes, function objects are first-class and possess a writable __dict__ attribute
(C) Only if decorated with @property
(D) Only in Python 2

90. What is the output of:\ndef f(): pass\nf.counter = 42\nprint(f.counter)

(A) 42
(B) AttributeError: 'function' object has no attribute 'counter'
(C) None
(D) TypeError

91. What does the functools.wraps decorator do when writing custom function decorators?

(A) Encrypts the function bytecode
(B) Preserves the original function's metadata (__name__, __doc__, __annotations__)
(C) Prevents recursion errors
(D) Speeds up execution by 2x

92. What is a Generator Function in Python?

(A) A function that compiles C libraries
(B) A function that contains the 'yield' keyword and returns an iterator yielding values on demand
(C) A function that generates random numbers
(D) A recursive function without base case

93. What happens when a generator function is invoked: gen = my_generator()?

(A) It executes the entire function immediately to completion
(B) It returns a generator object without executing any code until next() is called
(C) It raises a StopIteration exception
(D) It starts a background thread

94. What is the memory complexity of a generator producing 1 billion integers using yield?

(A) O(1) constant memory
(B) O(N) allocating 1 billion items in RAM
(C) O(log N)
(D) O(N^2)

95. What does the 'yield from' expression introduced in Python 3.3 do?

(A) Terminates the generator
(B) Delegates iteration directly to a sub-generator or iterable
(C) Yields an empty tuple
(D) Catches all exceptions in generator

96. What is the output of:\ndef flatten(nested):\n for item in nested:\n if isinstance(item, list):\n yield from flatten(item)\n else:\n yield item\nprint(list(flatten([1, [2, [3, 4]], 5])))

(A) [1, 2, 3, 4, 5]
(B) [[1, 2], [3, 4], 5]
(C) [1, [2, 3, 4], 5]
(D) TypeError

97. What is the Ackermann function A(m, n) famous for in theoretical computer science?

(A) It is an extremely fast sorting algorithm
(B) It is an example of a non-primitive recursive function that grows extraordinarily rapidly
(C) It computes prime numbers in O(1)
(D) It solves the traveling salesperson problem

98. What is the value of the Ackermann function A(1, 2)?

(A) 3
(B) 4
(C) 5
(D) 6

99. What happens when trying to evaluate Ackermann A(4, 2) in Python?

(A) Returns 65536 in 1 ms
(B) Immediately triggers RecursionError due to massive call stack expansion
(C) Returns 16
(D) Returns None

100. What is the role of a Trampoline in functional Python programming?

(A) A mechanism to bounce exceptions back to caller
(B) An iterative loop that repeatedly executes thunks to simulate tail-call optimization with O(1) stack depth
(C) A visual debugging tool in IDEs
(D) A garbage collection cycle

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Explain the Anatomy of a Python Function. Detail function definitions (def), parameter declarations, docstrings, return semantics, and the LEGB variable scope lookup rule.

Q2. Analyze Function Arguments in Python: Positional, Keyword, Default, Variable-Length (*args, **kwargs), Keyword-Only (*), and Positional-Only (/).

Q3. Explain the Mutable Default Argument Trap in Python. Detail why def f(lst=[]) causes state contamination across calls, and demonstrate the idiomatic None sentinel pattern.

Q4. Explain First-Class Functions and Higher-Order Functions in Python. Implement custom versions of map(), filter(), and reduce() with lambda expressions.

Q5. Explain Scope, Nested Functions, Closures, and the nonlocal Keyword in Python with memory activation models.

Q6. Explain Mathematical Functions in Python's standard math module. Detail trigonometry, logarithms, power, ceiling/floor, and special constants (pi, e, inf, nan).

Q7. Explain Recursion Fundamentals in Python: Base Cases, Recursive Steps, Call Stack Mechanics, and Stack Frame Activation Records.

Q8. Explain the Recursion Limit in Python. Detail RecursionError, sys.getrecursionlimit(), sys.setrecursionlimit(), and why Python avoids Tail Call Optimization (TCO).

Q9. Implement the Tower of Hanoi algorithm in Python using recursion. Explain the mathematical induction, move sequences, and O(2^N) complexity analysis.

Q10. Implement Binary Search recursively in Python. Detail subproblem partitioning, mid calculation, base cases, and compare with iterative implementation.

Q11. Explain Memoization and Dynamic Programming. Implement the Fibonacci sequence and compare naive recursive O(2^N), manual memoization O(N), and @lru_cache.

Q12. Implement Binary Exponentiation (Fast Power Algorithm) in Python using both recursive divide-and-conquer and iterative bit manipulation.

Q13. Implement a Recursive Merge Sort algorithm in Python. Analyze divide-and-conquer splitting, merging, stability, and O(N log N) asymptotic complexity.

Q14. Implement a Recursive QuickSort algorithm in Python with Hoare/Lomuto partitioning. Compare worst-case O(N^2) vs average-case O(N log N).

Q15. Explain Custom Function Decorators in Python. Detail how @wraps preserves metadata, and implement execution timers and input validation decorators.

Q16. Implement a Recursive Filesystem Directory Tree Walker in Python using os and recursion, calculating total disk usage and printing structured indentation.

Q17. Implement a Recursive Backtracking Maze Solver in Python using 2D grids, finding a path from start (0,0) to exit (M-1, N-1).

Q18. Explain Function Annotations and Type Hints (PEP 484) in Python. Contrast dynamic typing with static type checking using mypy.

Q19. Implement the N-Queens Puzzle in Python using recursive backtracking. Find all valid non-attacking queen configurations on an N x N chessboard.

Q20. Explain Generator Functions and the yield Keyword. Contrast generators with standard return functions in memory usage and streaming pipelines.

Q21. Implement a Recursive Flattening algorithm for arbitrarily nested lists of irregular depths in Python with type checking.

Q22. Implement a String Permutation and Combination generator in Python without external libraries using recursive backtracking and list swapping.

Q23. Implement a Recursive Palindrome Partitioning program in Python that decomposes a string into all possible palindromic substring sequences.

Q24. Implement an Expression Evaluator for Infix Arithmetic (+, -, *, /) in Python using recursive descent parsing or two stacks.

Q25. Explain Partial Function Application using functools.partial in Python. Contrast partial application with Currying with practical examples.

Q26. Implement a Trampoline pattern in Python to eliminate recursion stack overflow for deep tail-recursive algorithms.

Q27. Explain Function Introspection using the inspect and typing modules in Python with parameter signature extraction.

Q28. Implement a Recursive Sudoku Solver in Python using backtracking with constraint propagation.

Q29. Implement a Recursive Descent XML/HTML Tokenizer and Tag Balancer in Python using recursion and stack validation.

Q30. Implement the Ackermann Function in Python with recursion analysis, rapid growth metrics, and recursion limit protection.

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1:	[B]	**Q21:**	[C]	**Q41:**	[B]	**Q61:**	[B]	**Q81:**	[B]
Q2:	[C]	**Q22:**	[B]	**Q42:**	[B]	**Q62:**	[A]	**Q82:**	[A]
Q3:	[C]	**Q23:**	[A]	**Q43:**	[B]	**Q63:**	[B]	**Q83:**	[B]
Q4:	[A]	**Q24:**	[B]	**Q44:**	[A]	**Q64:**	[B]	**Q84:**	[B]
Q5:	[B]	**Q25:**	[B]	**Q45:**	[A]	**Q65:**	[B]	**Q85:**	[B]
Q6:	[B]	**Q26:**	[B]	**Q46:**	[A]	**Q66:**	[B]	**Q86:**	[B]
Q7:	[B]	**Q27:**	[B]	**Q47:**	[B]	**Q67:**	[A]	**Q87:**	[B]
Q8:	[A]	**Q28:**	[B]	**Q48:**	[A]	**Q68:**	[B]	**Q88:**	[B]
Q9:	[A]	**Q29:**	[B]	**Q49:**	[A]	**Q69:**	[A]	**Q89:**	[B]
Q10:	[C]	**Q30:**	[B]	**Q50:**	[A]	**Q70:**	[A]	**Q90:**	[A]
Q11:	[A]	**Q31:**	[A]	**Q51:**	[B]	**Q71:**	[A]	**Q91:**	[B]
Q12:	[B]	**Q32:**	[B]	**Q52:**	[B]	**Q72:**	[B]	**Q92:**	[B]
Q13:	[B]	**Q33:**	[A]	**Q53:**	[B]	**Q73:**	[A]	**Q93:**	[B]
Q14:	[C]	**Q34:**	[B]	**Q54:**	[B]	**Q74:**	[B]	**Q94:**	[A]
Q15:	[B]	**Q35:**	[B]	**Q55:**	[D]	**Q75:**	[A]	**Q95:**	[B]
Q16:	[B]	**Q36:**	[A]	**Q56:**	[B]	**Q76:**	[B]	**Q96:**	[A]
Q17:	[B]	**Q37:**	[A]	**Q57:**	[B]	**Q77:**	[B]	**Q97:**	[B]
Q18:	[B]	**Q38:**	[A]	**Q58:**	[A]	**Q78:**	[B]	**Q98:**	[B]
Q19:	[B]	**Q39:**	[A]	**Q59:**	[A]	**Q79:**	[A]	**Q99:**	[B]
Q20:	[B]	**Q40:**	[B]	**Q60:**	[A]	**Q80:**	[B]	**Q100:**	[B]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

SECTION B: DETAILED SOLUTIONS FOR SUBJECTIVE QUESTIONS function definitions (def), parameter declarations, docstrings, return semantics, and th

- Global / Builtin Access: O(1) hash table lookup in globals() and __builtins__.__dict__.

Solution to Question 2:

>>> # Solution to Question 2: Analyze Function Arguments in Python: Positional, Keyword, Default, Variable-Length (*args, **kwargs), Keyword-Only (*), and Positional

- Argument Binding: O(P + K) where P is positional count and K is keyword count.
- Space Complexity: O(len(args)) tuple + O(len(kwargs)) dict.

Solution to Question 3: Explain the Mutable Default Argument Trap in Python. Detail why def f(lst=[]) causes state contamination across calls, and demonstrate

>>P Solution Statement Question 4: Explain First-Class Functions and Higher-Order Functions in Python. Implement custom versions of map(), filter(), and reduce() with lar

```
# Filter odd numbers via my_filter
evens = list(my_filter(lambda x: x % 2 == 0, nums))
print(f"my_filter (evens): {evens}")

# Compute product via my_reduce
product = my_reduce(lambda acc, x: acc * x, nums, 1)
print(f"my_reduce (5!): {product}")

5. Complexity Analysis:
- my_map / my_filter Time Complexity: O(N) streaming evaluations.
- Space Complexity: O(1) lazy generator memory.
- my_reduce Time: O(N), Space: O(1) accumulator register.
```

Solution to Question 5: Explain Scope, Nested Functions, Closures, and the nonlocal Keyword in Python with memory activation models.

```
  def get_balance() -> float:
  return balance

  return deposit, withdraw, get_balance

  # Instantiate independent bank accounts
  dep, withdr, bal = make_bank_account("Aarav", 1000.0)
  print(f"Initial Balance: ${bal():.2f}")
  dep(500.0)
  print(f"After $500 Deposit: ${bal():.2f}")
  withdr(200.0)
  print(f"After $200 Withdrawal: ${bal():.2f}")

  # Introspecting Closure Cells
  print(f"Closure Cell Contents: {dep.__closure__[0].cell_contents}")
```

4. Complexity Analysis:
- Time Complexity: O(1) cell pointer dereferencing (DEREF opcode).
- Space Complexity: O(1) PyCellObject allocation per captured variable.

>>P Solution to Question 6: Explain Mathematical Functions in Python's standard math module. Detail trigonometry, logarithms, power, ceiling/floor, and special co

```
print(f"Angle: {angle_deg} deg = {rad:.4f} rad")
print(f"sin(45 deg) = {math.sin(rad):.4f} (Expected: {1/math.sqrt(2):.4f})")
print(f"Hypotenuse (3, 4): {math.hypot(3, 4)}")

# Logarithmic calculations
val = 1000.0
print(f"log10({val}) = {math.log10(val):.2f}")
print(f"log2(1024) = {math.log2(1024):.2f}")

# Ceiling and Floor
x = 7.35
print(f"ceil({x}) = {math.ceil(x)} | floor({x}) = {math.floor(x)}")

# GCD and Factorial
print(f"math.gcd(84, 180) = {math.gcd(84, 180)}")
print(f"math.factorial(6) = {math.factorial(6)}")

scientific_calculator_demo()
```

4. Complexity Analysis:
- Time Complexity: O(1) single-cycle hardware FPU / C-lib execution.
- Space Complexity: O(1) register operations.

>> Solution to Question 7: Main Recursion Fundamentals in Python: Base Cases, Recursive Steps, Call Stack Mechanics, and Stack Frame Activation Records

```
if n <= 1:
print(f"{indent}<- BASE CASE REACHED: factorial({n}) returns 1")
return 1

# Recursive Step
sub_res = factorial_traced(n - 1)
result = n * sub_res
print(f"{indent}<- POP frame: factorial({n}) = {n} * {sub_res} = {result}")
return result

print("Execution Trace of factorial_traced(3):")
final_ans = factorial_traced(3)
print("Final Returned Factorial:", final_ans)

4. Complexity Analysis:
- Time Complexity: O(N) recursive calls.
- Space Complexity: O(N) peak call stack depth.
```

>>Solution to Question 8: Explain the Recursion Limit in Python, Detail RecursionError, sys.getrecursionlimit(), sys.setrecursionlimit(), and why Python avoids Ta

```
try:
return test_recursion_depth(n + 1)
except RecursionError:
return n

max_depth_reached = test_recursion_depth(1)
print(f"RecursionError triggered at call depth: {max_depth_reached}")
```

5. Complexity Analysis:
- Space Complexity: O(Recursion_Limit) bounded stack memory.

>> Solution to Question 9: Implement the Tower of Hanoi algorithm in Python using recursion. Explain the mathematical induction, move sequences, and O(2^N) c

```
# Execution for N = 3 disks (Total moves = 2^3 - 1 = 7)
disks = 3
moves = solve_tower_of_hanoi(disks, 'A', 'C', 'B')
print(f"Tower of Hanoi Solution for N={disks} Disks ({len(moves)} total moves):")
for idx, move in enumerate(moves, start=1):
print(f" Step {idx}: {move}")
```

4. Complexity Analysis:
- Time Complexity: O(2^N) exponential time.
- Auxiliary Space Complexity: O(N) stack frame depth.

>> Solution to Question 10: Implement Binary Search recursively in Python. Detail subproblem partitioning, mid calculation, base cases, and compare with iterative

```
target2 = 99

idx1 = binary_search_recursive(sorted_data, target1)
idx2 = binary_search_recursive(sorted_data, target2)

print(f"Dataset: {sorted_data}")
print(f"Search for {target1} -> Index: {idx1} (Value: {sorted_data[idx1]})")
print(f"Search for {target2} -> Index: {idx2} (Not Found)")
```

4. Complexity Analysis:
- Time Complexity: O(log2 N) logarithmic time (array halved at each step).
- Space Complexity: O(log2 N) call stack frames (versus O(1) for iterative binary search).

>> Solution to Question 11: Explain Memoization and Dynamic Programming. Implement the Fibonacci sequence and compare naive recursive O(2^N), manual me

```
start_t = time.perf_counter()
ans_manual = fib_manual_memo(n_target)
time_manual = (time.perf_counter() - start_t) * 1000

# LRU Cache Benchmark
start_t = time.perf_counter()
ans_lru = fib_lru_cached(n_target)
time_lru = (time.perf_counter() - start_t) * 1000

print(f"Fibonacci({n_target}) = {ans_manual}")
print(f"Manual Memoization Time: {time_manual:.4f} ms")
print(f"functools.lru_cache Time: {time_lru:.4f} ms")
print(f"Cache Statistics: {fib_lru_cached.cache_info()}")
```

4. Complexity Analysis:
- Naive Time Complexity: O(2^N) exponential.
- Memoized Time Complexity: O(N) linear time (each state computed exactly once).
- Space Complexity: O(N) memory for cache table and stack depth.

>>Solution to Question 12: Implement Binary Exponentiation (Fast Power Algorithm) in Python using both recursive divide-and-conquer and iterative bit manipulati

```
x = 1.0 / x
n = -n

result = 1.0
current_base = x

while n > 0:
# If current least significant bit is 1
if n & 1:
result *= current_base
current_base *= current_base # Square base
n >>= 1 # Shift bits right

return result

# Verification
base = 2.0
exponent = 20

res_rec = FastPower.recursive_pow(base, exponent)
res_iter = FastPower.iterative_bit_pow(base, exponent)
expected = 2 ** 20

print(f"{base}^{exponent} via Recursive: {res_rec}")
print(f"{base}^{exponent} via Iterative: {res_iter} (Matches 2^20: {res_iter == expected})")

3. Complexity Analysis:
- Time Complexity: O(log2 N) multiplications (only ~20 multiplications for n = 1,000,000).
- Recursive Space: O(log2 N) stack frames.
- Iterative Space: O(1) strictly constant space.
```

>>P Solution Statement Question 13: Implement a Recursive Merge Sort algorithm in Python. Analyze divide-and-conquer splitting, merging, stability, and O(N log N) asymp

```
mid = len(arr) // 2
# Divide
left_sorted = merge_sort(arr[:mid])
right_sorted = merge_sort(arr[mid:])

# Combine / Merge
merged = []
i = j = 0

while i < len(left_sorted) and j < len(right_sorted):
if left_sorted[i] <= right_sorted[j]:
merged.append(left_sorted[i])
i += 1
else:
merged.append(right_sorted[j])
j += 1

# Append any remaining elements
merged.extend(left_sorted[i:])
merged.extend(right_sorted[j:])
return merged

# Verification
sample = [38, 27, 43, 3, 9, 82, 10]
sorted_sample = merge_sort(sample)
print(f"Unsorted List: {sample}")
print(f"Sorted List: {sorted_sample}")

4. Complexity Analysis:
- Time Complexity: Recurrence T(N) = 2*T(N/2) + O(N) -> O(N log2 N) by Master Theorem.
- Auxiliary Space Complexity: O(N) memory to create merged subarrays.
```

>>Solution to Question 14: Implement a Recursive QuickSort algorithm in Python with Hoare/Lomuto partitioning. Compare worst-case O(N^2) vs average-case O

```
    return arr

    # Choose midpoint as pivot to avoid O(N^2) on sorted lists
    pivot = arr[len(arr) // 2]

    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quicksort_idiomatic(left) + middle + quicksort_idiomatic(right)

    # Verification
    dataset = [33, 10, 55, 71, 29, 10, 14, 5, 88]
    sorted_data = quicksort_idiomatic(dataset)
    print(f"Original: {dataset}")
    print(f"Sorted: {sorted_data}")

    4. Complexity Analysis:
    - Average Time: O(N log N).
    - Worst Case Time: O(N^2).
    - Space Complexity: O(N) memory for comprehension partitions.
```

>>Solution to Question 15: Explain Custom Function Decorators in Python. Detail how @wraps preserves metadata, and implement execution timers and input val

```
print(f"[BENCHMARK] '{func.__name__}' executed in {elapsed_ms:.4f} ms")
return result
return wrapper

def positive_args_only(func):
"""Validates that all numeric arguments are positive."""
@wraps(func)
def wrapper(*args, **kwargs):
for arg in args:
if isinstance(arg, (int, float)) and arg < 0:
raise ValueError(f"Invalid Argument: Negative value {arg} rejected.")
return func(*args, **kwargs)
return wrapper

@timing_decorator
@positive_args_only
def compute_sum_of_squares(n: int) -> int:
"""Compute sum of squares up to n."""
return sum(x ** 2 for x in range(n + 1))

# Execution
res = compute_sum_of_squares(50_000)
print(f"Result: {res} | Function Name Preserved: {compute_sum_of_squares.__name__}")

# Validation Failure Test
try:
compute_sum_of_squares(-10)
except ValueError as err:
print(f"Validation Error Caught: {err}")

4. Complexity Analysis:
- Decorator Overhead: O(1) single wrapper redirection per call.
```

>> Solution to Question 16: Implement a Recursive Filesystem Directory Tree Walker in Python using os and recursion, calculating total disk usage and printing str

```
dir_count = 1 # Count current directory

try:
entries = os.listdir(path)
except PermissionError:
print(f"{indent} [ACCESS DENIED]")
return 0, 0, 0

for entry in sorted(entries):
full_path = os.path.join(path, entry)

if os.path.isdir(full_path):
sub_size, sub_files, sub_dirs = walk_directory_tree(full_path, depth + 1)
total_size += sub_size
file_count += sub_files
dir_count += sub_dirs
else:
try:
sz = os.path.getsize(full_path)
except OSError:
sz = 0
total_size += sz
file_count += 1
print(f"{indent} - {entry} ({sz:,} bytes)")

return total_size, file_count, dir_count

# Simulation on a safe system path
sample_target = "C:/FINAL PrOJECT/cse unit folder/python/u1"
if os.path.exists(sample_target):
total_bytes, total_files, total_dirs = walk_directory_tree(sample_target)
print("=" * 60)
print(f"Summary: {total_dirs} Dirs, {total_files} Files, Total Size: {total_bytes / 1024:.2f}
KB")

3. Complexity Analysis:
- Time Complexity: O(Total Inodes / Files) visiting each filesystem entry once.
- Space Complexity: O(Depth) directory tree recursion stack.
```

>>Solution to Question 17

Solution to Question 17: Implement a Recursive Backtracking Maze Solver in Python using 2D grids, finding a path from start (0,0) to exit (M-1, N-1).

If any direction leads to target -> Return True.
4. If all directions fail: Unmark cell (backtrack) and return False.

2. Python Implementation:

```
class MazeSolver:
@staticmethod
def solve(maze):
rows = len(maze)
cols = len(maze[0])
path = []

def backtrack(r, c):
if not (0 <= r < rows and 0 <= c < cols):
return False
if maze[r][c] != 0:
return False # Wall or already visited

maze[r][c] = 2 # Mark cell as part of solution path
path.append((r, c))

# Target reached
if r == rows - 1 and c == cols - 1:
return True

# Explore Down, Right, Up, Left
if (backtrack(r + 1, c) or
backtrack(r, c + 1) or
backtrack(r - 1, c) or
backtrack(r, c - 1)):
return True

# Backtrack
path.pop()
maze[r][c] = 3 # Mark as dead end
return False

success = backtrack(0, 0)
return success, path

# Verification
test_maze = [
[0, 1, 0, 0],
[0, 0, 0, 1],
[1, 0, 1, 0],
[0, 0, 0, 0]
]

solved, solution_path = MazeSolver.solve(test_maze)
print(f"Maze Solvable? {solved}")
print(f"Path Coordinates ({len(solution_path)} steps): {solution_path}")
```

3. Complexity Analysis:
- Time Complexity: O(4^(M*N)) worst case; bounded by O(M * N) with visited memoization.
- Space Complexity: O(M * N) call stack and path buffer.

Solution to Question 18:

>>> Explain Function Annotations and Type Hints (PEP 484) in Python. Contrast dynamic typing with static type checking using mypy.

Solution to Question 19:

```
# Implement the N-Queens Puzzle in Python using recursive backtracking. Find all valid non-attacking queen configurations on an N x N
```

```
    neg_diag.remove(r - c)
    board[r][c] = "."

    backtrack(0)

    # Verification for N = 4 Queens (2 distinct solutions exist)
    solver_4 = NQueensSolver(4)
    print(f"Total Unique Solutions for N=4 Queens: {len(solver_4.solutions)}")
    for idx, sol in enumerate(solver_4.solutions, start=1):
    print(f"--- Solution #{idx} ---")
    for row in sol:
    print(" ", row)
```

4. Complexity Analysis:
- Time Complexity: O(N!) factorial search space bounded by pruning.
- Space Complexity: O(N) call stack and board state.

>> Solution to Question 20: Explain Generator Functions and the yield Keyword. Contrast generators with standard return functions in memory usage and streamin

```
# Data Pipeline Chaining
def evens_filter(stream):
for val in stream:
if val % 2 == 0:
yield val

even_fibs = [next(evens_filter(fib_gen)) for _ in range(4)]
print(f"Next 4 Even Fibonacci numbers: {even_fibs}")
```

4. Complexity Analysis:
- Time Complexity: O(1) per yielded element.
- Space Complexity: O(1) strictly constant memory frame.

Solution to Question 21:

>>> Problem Statement: Implement a Recursive Flattening algorithm for arbitrarily nested lists of irregular depths in Python with type checking.

```
print(f"Flattened List: {result_flat}")
print(f"Generator Match: {result_flat == gen_flat}")

4. Complexity Analysis:
- Time Complexity: O(N) where N is total scalar leaf elements.
- Space Complexity: O(Depth) recursion stack depth.
```

>>Solution to Question 28: Implement a Recursive Permutation and Combination generator in Python that efficiently handles large inputs using backtracking and memoization.

1. Problem Context:
Given a string s = 'aab', partition s such that every substring of the partition is a palindrome. Return all possible palindrome partitioning configurations (e.g., [['a', 'a', 'b'], ['aa', 'b']]).

2. Backtracking Strategy:
1. Base Case: If start index reaches len(s), add current partition snapshot to results.
2. For end in range(start, len(s)):
Extract substring = s[start : end + 1].
If substring is a palindrome (substring == substring[::-1]):
- Append to current partition.
- Recurse for end + 1.
- Pop substring to backtrack.

3. Python Implementation:

```
def partition_palindromes(s: str) -> list:
results = []
current_path = []

def is_palindrome(sub: str) -> bool:
return sub == sub[::-1]

def backtrack(start: int):
if start == len(s):
results.append(list(current_path))
return

for end in range(start, len(s)):
substring = s[start : end + 1]
if is_palindrome(substring):
current_path.append(substring)
backtrack(end + 1)
current_path.pop() # Backtrack

backtrack(0)
return results

# Verification
target = "aab"
partitions = partition_palindromes(target)
print(f"Palindrome Partitions for '{target}': {partitions}")
```

4. Complexity Analysis:
- Time Complexity: O(N * 2^N) exponential combinations.
- Space Complexity: O(N) recursive call stack.

>>Solution to Question 24: Implement an Expression Evaluator for Infix Arithmetic (+, -, *, /) in Python using recursive descent parsing or two stacks.

- Term: Factor (('*' | '/') Factor)*
- Factor: Number | '(' Expression ')'

2. Python Implementation:

```
class SimpleCalculatorParser:
def __init__(self, expression: str):
self.tokens = expression.replace(" ", "")
self.pos = 0

def parse(self) -> float:
return self._expr()

def _expr(self) -> float:
val = self._term()
while self.pos < len(self.tokens) and self.tokens[self.pos] in ('+', '-'):
op = self.tokens[self.pos]
self.pos += 1
term_val = self._term()
val = val + term_val if op == '+' else val - term_val
return val

def _term(self) -> float:
val = self._factor()
while self.pos < len(self.tokens) and self.tokens[self.pos] in ('*', '/'):
op = self.tokens[self.pos]
self.pos += 1
factor_val = self._factor()
val = val * factor_val if op == '*' else val / factor_val
return val

def _factor(self) -> float:
if self.tokens[self.pos] == '(':
self.pos += 1 # Skip '('
val = self._expr()
self.pos += 1 # Skip ')'
return val

# Parse number
start = self.pos
while self.pos < len(self.tokens) and (self.tokens[self.pos].isdigit() or self.tokens[self.pos]
== '.'):
self.pos += 1
return float(self.tokens[start:self.pos])

# Verification
calc = SimpleCalculatorParser("3 + 5 * (10 - 2) / 4")
calc_result = calc.parse()
print(f"Parsed '3 + 5 * (10 - 2) / 4' = {calc_result} (Expected: {3 + 5 * (10 - 2) / 4})")
```

3. Complexity Analysis:
- Time Complexity: O(N) single linear pass over tokens.
- Space Complexity: O(Depth) recursion stack for nested parentheses.

>>Solution Question 28 Partial Function Application Python functools Stack Overflow recursion with practical examples.

```
  return res

  def factorial_tail_thunk(n: int, acc=1):
  if n <= 1:
  return acc
  # Return a lambda thunk instead of recursing directly
  return lambda: factorial_tail_thunk(n - 1, acc * n)

  # Run via Trampoline
  big_n = 500
  result = Trampoline.call(factorial_tail_thunk, big_n)
  print(f"Factorial({big_n}) computed with Trampoline without StackOverflow!")
  print(f"Digits in {big_n}!: {len(str(result))} digits")

  3. Complexity Analysis:
  - Time Complexity: O(N) linear loop.
  - Stack Space: O(1) strictly constant stack depth!
```

---

2. Python Implementation:

```
def solve_sudoku_grid(board: list) -> bool:
def is_valid(r, c, ch):
for i in range(9):
if board[r][i] == ch or board[i][c] == ch:
return False
if board[3 * (r // 3) + i // 3][3 * (c // 3) + i % 3] == ch:
return False
return True

def backtrack():
for r in range(9):
for c in range(9):
if board[r][c] == '.':
for d in map(str, range(1, 10)):
if is_valid(r, c, d):
board[r][c] = d
if backtrack():
return True
board[r][c] = '.' # Backtrack
return False
return True

return backtrack()

# Verification
sample = [
["5","3",".",".","7",".",".",".",".","."],
["6",".",".","1","9","5",".",".",".","."],
[".","9","8",".",".",".",".",".","6","."],
["8",".",".",".","6",".",".",".",".","3"],
["4",".",".","8",".","3",".",".",".","1"],
["7",".",".",".",".","2",".",".",".",".","6"],
[".","6",".",".",".",".",".","2","8","."],
[".",".",".","4","1","9",".",".",".","5"],
[".",".",".",".",".","8",".",".","7","9"]
]

solve_sudoku_grid(sample)
print("Solved Sudoku Row 0:", sample[0])
```

3. Complexity Analysis:
- Time Complexity: O(9^(Empty_Cells)) exponential backtracking.
- Space Complexity: O(81) recursion depth.

Solution to Question 29:

>> Problem Statement: Implement a Recursive Descent XML/HTML Tokenizer and Tag Balancer in Python using recursion and stack validation.

2. Python Implementation:

```
import re

def is_markup_balanced(html_str: str) -> bool:
tag_regex = re.compile(r'</?([a-zA-Z0-9]+)>')
tags = tag_regex.findall(html_str)
raw_tokens = tag_regex.finditer(html_str)

stack = []
for match in raw_tokens:
full_tag = match.group(0)
tag_name = match.group(1)

if full_tag.startswith("</"): # Closing tag
if not stack or stack[-1] != tag_name:
return False
stack.pop()
else: # Opening tag
stack.append(tag_name)

return len(stack) == 0

# Verification
valid_html = "<html><body><h1>Title</h1><p>Body</p></body></html>"
invalid_html = "<html><body><h1>Title</p></h1></body></html>"

print(f"Is valid_html balanced? {is_markup_balanced(valid_html)}")
print(f"Is invalid_html balanced? {is_markup_balanced(invalid_html)}")
```

3. Complexity Analysis:
- Time Complexity: O(N) regex scan.
- Space Complexity: O(Depth) tag stack.

>>Solution to Question 30: Implement the Ackermann Function in Python with recursion analysis, rapid growth metrics, and recursion limit protection.

```
# Verification of standard computable values
print(f"A(0, 5) = {ackermann(0, 5)}")
print(f"A(1, 2) = {ackermann(1, 2)}")
print(f"A(2, 3) = {ackermann(2, 3)}")
print(f"A(3, 3) = {ackermann(3, 3)} (Value = 61)")
```

3. Complexity Analysis:
- Growth Rate: Hugely outpaces exponential functions (Ackermann hierarchy).
- Space Complexity: Bounded by recursion depth limits.
